

ASSESSMENT WORKBOOK
Semester Genap T.A. 2025-2026

Program Studi D4 Teknik Informatika – Sekolah Vokasi
Jl. Sari Asih No 54, Sarijadi, Kota Bandung, 40151

Nama Mahasiswa	:	Nanda Septiana Ramadhani
NPM	:	714240033
Mata Kuliah	:	Rekayasa Perangkat Lunak II

1. Deskripsi Aplikasi SupplierHub

SupplierHub adalah aplikasi penyedia barang (Provider) dalam ekosistem UMKM digital yang berfungsi menghubungkan supplier atau distributor besar dengan para pelaku UMKM di Marketplace. Aplikasi ini bertindak sebagai jembatan Business-to-Business (B2B) yang memastikan ketersediaan rantai pasok barang tetap terjaga.

Aplikasi ini dibatasi hanya untuk interaksi antar penyedia dan penjual (UMKM), sehingga tidak mengelola penjualan langsung ke konsumen akhir, guna menjaga fokus pada efisiensi distribusi barang di dalam ekosistem.

2. Analisis Transaksi End-to-End SupplierHub

Sebagai developer utama SupplierHub, berikut analisis proses transaksi ekonomi utama (pengadaan barang oleh UMKM/Marketplace) pada sistem kami:

1) Input Utama yang Diterima Aplikasi:

SupplierHub menerima input dari dua entitas utama: **Marketplace (PasarKita)** sebagai pemesan barang dan **Admin Supplier** sebagai pengelola stok. Namun, dalam konteks transaksi ekonomi utama (Order Fulfillment), input yang paling krusial adalah:

- a. Data Pesanan Transaksional: Berupa request dari Marketplace yang berisi detail pesanan seperti ID pesanan, daftar produk beserta jumlahnya, identitas pembeli (UMKM), serta lokasi tujuan pengiriman.
- b. Metadata Keamanan dan Otorisasi: Berupa token JWT (JSON Web Token) pada header permintaan untuk validasi identitas serta timestamp untuk menjamin sinkronisasi waktu transaksi di seluruh ekosistem.
- c. Input Manajemen Inevntaris: Data pembaruan stok dan katalog dari sisi Admin Supplier yang digunakan untuk memastikan ketersediaan barang sebelum dipublikasikan ke aplikasi Marketplace.

2) API yang Perlu Dipanggil ke Sistem Lain:

- a. **(POST /logistics/request_shipping):** untuk mendaftarkan instruksi pengiriman dan mendapatkan kalkulasi ongkos kirim.
- b. **POST /smartbank/payment:** untuk menginisiasi penagihan melalui API SmartBank

3) Data yang Dikirim dan Diterima:

Saat berinteraksi dengan LogistiKita, SupplierHub mengirimkan parameter berupa ID pesanan, alamat asal gudang, alamat tujuan UMKM, dan berat total barang.

Sebagai timbal balik, sistem logistik akan mengirimkan data berupa nomor resi (*shipping_id*), biaya pengiriman (*shipping_cost*), serta estimasi waktu pengiriman yang akan diteruskan kembali ke pembeli.

Pada interaksi dengan SmartBank, SupplierHub mengirimkan rincian identitas pembayar (*from_user*), identitas penerima (*to_user*), total nominal transaksi, serta metadata tambahan untuk kebutuhan pencatatan ledger. Respon yang diharapkan dari SmartBank adalah status final transaksi (berhasil atau gagal), ID referensi transaksi sebagai bukti sah, serta *timestamp* penyelesaian untuk menjamin integritas data pada catatan keuangan kedua belah pihak.

4) Mekanisme Validasi JWT/token:

Keamanan transaksi pada SupplierHub dikelola melalui mekanisme otentikasi berbasis token JWT yang wajib disertakan pada setiap header permintaan. Meskipun API Gateway bertindak sebagai garda terdepan dalam melakukan validasi awal terhadap validitas tanda tangan (*signature*) token, SupplierHub menerapkan *middleware* internal untuk melakukan dekode dan verifikasi klaim data di dalam token tersebut. Proses ini mencakup pemeriksaan masa berlaku token melalui klaim **exp** (*expiration time*) untuk mencegah penggunaan token lama yang telah kadaluwarsa serta verifikasi klaim **iss** (*issuer*) guna memastikan token berasal dari otoritas identitas yang sah di dalam ekosistem.

Setelah integritas token terverifikasi, SupplierHub mengekstraksi informasi *user_id* dan *role* untuk melakukan otorisasi tingkat lanjut. Hal ini dilakukan untuk memastikan bahwa request pemrosesan pesanan hanya datang dari node Marketplace yang terpercaya, atau akses manajemen stok hanya dilakukan oleh pengguna dengan level akses Admin Supplier. Jika token tidak valid, dimanipulasi, atau tidak memiliki izin akses yang sesuai, sistem akan secara otomatis menolak permintaan dengan mengirimkan respon kode error 401 Unauthorized atau 403 Forbidden sebelum logika bisnis sempat dijalankan, sehingga menjamin keamanan sumber daya aplikasi dari akses yang tidak sah.

5) Risiko Inkonsistensi Data:

Risiko paling nyata terjadi pada ketidakselarasan antara jumlah stok di database internal dengan status transaksi di SmartBank; misalnya, stok barang sudah terlanjur dikurangi saat *checkout* dimulai, namun pembayaran di SmartBank gagal atau mengalami *timeout*. Kondisi ini dapat menyebabkan "stok menggantung" (stok berkurang padahal barang tidak terjual), yang jika tidak ditangani dengan mekanisme *rollback* otomatis, akan merugikan supplier karena hilangnya potensi penjualan ke pembeli lain.

Selain itu, inkonsistensi dapat muncul pada status pengiriman di LogistiKita yang tidak terupdate ke SupplierHub maupun Marketplace akibat gangguan koneksi saat pemanggilan API. Risiko *race condition* juga menghantui jika ada dua permintaan transaksi untuk produk yang sama dalam waktu yang hampir bersamaan; tanpa mekanisme *locking* yang tepat, sistem mungkin akan mengizinkan pemesanan melebihi jumlah stok fisik yang tersedia (*overselling*).

6) Dampak Kegagalan Aplikasi Lain:

- a. Jika Marketplace mengalami kegagalan, SupplierHub secara otomatis akan kehilangan sumber *traffic* dan pesanan, mengakibatkan tumpukan stok yang tidak bergerak dan berhentinya perputaran uang bagi para supplier.
- b. Kegagalan pada LogistiKita akan berdampak pada ketidakmampuan sistem untuk memproses *fulfilment*. Pesanan mungkin sudah dibayar dan stok sudah dikurangi, namun ketiadaan layanan logistik menyebabkan barang tertahan di gudang, yang jika berlangsung lama akan merusak tingkat kepercayaan UMKM serta menurunkan *velocity of money* dalam sistem.
- c. Dampak yang paling kritis terjadi apabila SmartBank mengalami kegagalan operasional, karena SmartBank merupakan otoritas tunggal keuangan di ekosistem ini. Tanpa respon dari SmartBank, SupplierHub tidak dapat memvalidasi apakah dana pembeli mencukupi atau transaksi pembayaran telah berhasil, sehingga sistem tidak akan berani mengeksekusi pengiriman barang untuk menghindari kerugian materil.
- d. Selain itu, kegagalan pada API Gateway sebagai mediator komunikasi akan menyebabkan isolasi sistem secara total. Meskipun masing-masing aplikasi berjalan normal, mereka tidak dapat saling bertukar data, yang pada akhirnya memicu kegagalan berantai (*cascade failure*) di mana seluruh aktivitas ekonomi digital UMKM terhenti sepenuhnya.

7) Strategi Agar Sistem Tetap Robust:

- a. Penerapan mekanisme **Idempotency** pada seluruh endpoint pemrosesan pesanan. Setiap permintaan transaksi wajib menyertakan **idempotency_key** guna memastikan bahwa jika terjadi pengiriman ulang (*retry*) akibat *timeout* jaringan, SupplierHub tidak akan melakukan pengurangan stok ganda atau membuat instruksi pembayaran duplikat ke SmartBank.
- b. Sistem menggunakan pola **Circuit Breaker** pada integrasi API eksternal. Jika SmartBank atau LogistiKita memberikan respon lambat atau error secara berturut-turut, SupplierHub akan secara otomatis memutus sementara aliran permintaan ke layanan tersebut dan memberikan respon *fallback* yang informatif kepada pengguna, alih-alih membiarkan aplikasi hang atau kehabisan *resource*.
- c. Strategi lain berfokus pada integritas data melalui mekanisme **Stock Reservation** dengan masa kedaluwarsa singkat. Saat Marketplace melakukan *checkout*, stok tidak langsung dikurangi secara permanen melainkan "dikunci" untuk sementara waktu hingga konfirmasi pembayaran diterima dari SmartBank; jika dalam jangka waktu tertentu pembayaran tidak terverifikasi, stok akan dilepaskan kembali secara otomatis.
- d. SupplierHub menerapkan pencatatan log yang komprehensif pada setiap tahapan transaksi dan menggunakan mekanisme *background job* untuk melakukan sinkronisasi ulang secara periodik terhadap transaksi yang statusnya masih menggantung. Hal ini memastikan bahwa meskipun terjadi gangguan sementara pada salah satu node, SupplierHub tetap mampu melakukan pemulihan mandiri (*self-healing*) dan menjaga aliran ekonomi tetap stabil tanpa memicu kegagalan sistemik di seluruh ekosistem UMKM.

3. Analisis Strategi Menghadapi Lonjakan Transaksi

Analisis respons aplikasi terhadap lonjakan transaksi dan gangguan sistem mitra (SmartBank delay, LogistiKita delay, Marketplace surge):

1) Konsistensi Transaksi Ekonomi

SupplierHub menerapkan pola **Saga Pattern (Orchestration)**. Transaksi tidak dianggap "Selesai" jika salah satu layanan (SmartBank/LogistiKita) belum memberikan konfirmasi final. Mengingat SmartBank mengalami delay, SupplierHub akan menyimpan status transaksi sebagai **PENDING_PAYMENT**. Jika dalam batas waktu tertentu SmartBank tidak memberikan validasi, sistem akan melakukan *self-correction* dengan membatalkan pesanan di sisi SupplierHub agar tidak ada dana atau barang yang menggantung tanpa status hukum yang jelas dalam ekosistem.

2) Pencegahan Double Transaction

SupplierHub mewajibkan setiap *request* dari Marketplace menyertakan **Idempotency Key** unik. Jika Marketplace mengirimkan ulang *request* yang sama (akibat *retry* otomatis atau user menekan tombol berkali-kali), sistem akan mengenali kunci tersebut dan mengembalikan respon yang sama tanpa memproses ulang logika bisnis atau memanggil API SmartBank kembali. Hal ini menjamin bahwa satu pesanan hanya akan menghasilkan satu catatan transaksi di database SupplierHub.

3) Pencegahan Pengurangan Stok Palsu

SupplierHub menggunakan mekanisme **Atomic Stock Reservation**. Begitu pesanan masuk, stok dikurangi secara atomik di database. Namun, status stok ini adalah **RESERVED**. Jika validasi pembayaran dari SmartBank gagal atau mengalami *timeout* melampaui ambang batas, sistem akan menjalankan *background worker* untuk melakukan **Stock Release** (mengembalikan stok ke jumlah semula). Ini memastikan stok tidak berkurang secara permanen jika transaksi tidak benar-benar terjadi secara finansial.

4) Skalabilitas Sistem

Untuk menangani lonjakan beban, SupplierHub memisahkan proses sinkron pendaftaran pesanan dengan proses asinkron komunikasi ke sistem lain menggunakan **Message Queue**. Ketika Marketplace melakukan checkout, SupplierHub hanya mencatat pesanan dan segera memberikan respon "Diproses". Komunikasi ke SmartBank dan LogistiKita dilakukan oleh worker secara asinkron. Hal ini mencegah *bottleneck* pada *thread* utama aplikasi, sehingga sistem tetap dapat menerima ribuan request masuk meskipun sistem mitra sedang lambat merespon.

5) Feedback User yang Jelas

Alih-alih menampilkan pesan error saat LogistiKita atau SmartBank delay, SupplierHub akan mengirimkan status transisi yang informatif ke Marketplace, seperti: "*Pesanan Diterima, Menunggu Validasi Pembayaran*" atau "*Menghitung Estimasi Ongkir*". Jika terjadi kegagalan sinkronisasi ongkir dari LogistiKita, sistem akan memberikan nilai estimasi sementara dengan *disclaimer* atau meminta user menunggu sejenak tanpa memutuskan sesi transaksi, sehingga user tidak merasa sistem sedang rusak.

6) Pencegahan Cascade Failure

SupplierHub mengimplementasikan **Circuit Breaker** pada setiap integrasi API. Jika SmartBank atau LogistiKita gagal merespon dalam 5 detik selama beberapa kali percobaan, "saklar" akan terbuka dan SupplierHub akan berhenti mencoba menghubungi layanan tersebut untuk sementara waktu. Sebagai gantinya, sistem akan memberikan respon *fallback* (misal: menolak pesanan baru secara sopan atau masuk ke mode antrean). Ini mencegah tumpukan *request* yang menggantung di memori SupplierHub, sehingga aplikasi kita tidak ikut tumbang akibat kegagalan sistem mitra.

Berikut adalah identifikasi komponen kritis dan strategi teknis dalam aplikasi SupplierHub:

1) Komponen Paling Kritis

Inventory Manager (Stock Service) dan **External Integration Gateway**. Inventory Manager bertanggung jawab atas integritas data barang yang terbatas. Jika komponen ini gagal, akan terjadi *overselling* yang merusak reputasi supplier. Sementara External Integration Gateway menjadi kritis karena ia mengelola *Circuit Breaker* dan *Retry Logic* yang menjaga SupplierHub agar tetap hidup meskipun SmartBank atau LogistiKita sedang mengalami gangguan.

2) EndPoint/API yang Diprioritaskan

Endpoint yang harus diprioritaskan adalah **POST /supplier/order_fulfillment** (penerimaan pesanan) dan **GET /supplier/inventory/check**. Prioritas diberikan pada penerimaan pesanan agar aktivitas ekonomi dari Marketplace tetap terserap ke dalam antrian sistem, serta pengecekan stok agar user mendapatkan informasi ketersediaan yang akurat di tengah lonjakan transaksi.

3) Log yang Wajib Dicatat:

- a. **Transaction Lifecycle Log:** Mencatat setiap perubahan status pesanan (Created -> Reserved -> Paid -> Shipped).
- b. **External API Latency & Error Log:** Mencatat waktu respon dan kode error dari SmartBank dan LogistiKita untuk memicu *Circuit Breaker*.
- c. **Idempotency Log:** Mencatat penggunaan kunci idempotensi untuk mendeteksi percobaan transaksi ganda.
- d. **Stock Audit Trail:** Mencatat setiap penambahan atau pengurangan stok secara detail (siapa, kapan, dan alasannya).

4) Implementasi

Prinsip Dependency Inversion (DIP) memungkinkan logika bisnis SupplierHub tidak bergantung langsung pada implementasi API SmartBank. Kita menggunakan *interface* (abstraction) sehingga jika SmartBank berubah atau diganti, kode inti tetap stabil.

Prinsip Single Responsibility (SRP) memastikan kelas yang mengelola stok tidak terganggu oleh masalah koneksi pembayaran.

Clean Architecture memfasilitasi pemisahan antara "Use Case" (misal: **ProcessOrder**) dengan "External Agencies" (Logistik/Bank), sehingga kita dapat dengan mudah menambahkan mekanisme *fallback* atau *mocking* saat melakukan pengujian skenario kegagalan sistem tanpa merusak logika bisnis utama.